

ATTENTION AS A HYPERNETWORK

Simon Schug

ETH Zürich

sschug@ethz.ch

Seijin Kobayashi

ETH Zürich

seijink@ethz.ch

Yassir Akram

ETH Zürich

yakram@ethz.ch

João Sacramento[†]

Google, Paradigms of Intelligence Team

joaosacramento@google.com

Razvan Pascanu[†]

Google DeepMind

razp@google.com

Our **main contributions** can be summarized as follows:

- We reformulate standard multi-head attention from a hypernetwork perspective, revealing that network computations are specified by a low dimensional latent code.
- We show that scaling up model size and data enables transformers to compositionally generalize on abstract reasoning tasks and leads to a structured latent space that is predictive of network function.
- To test the hypothesis that the hypernetwork mechanism supports compositionality, we introduce a simple modification to multi-head linear attention that makes the value network nonlinear without introducing additional parameters, finding that it improves compositional generalization.
- We introduce SRAVEN, a challenging symbolic, abstract reasoning task whose difficulty can be controlled parametrically and which offers precise control over the problem compositions encountered during training in order to test for compositional generalization.

A *hypernetwork* is a neural network $h(\mathbf{z}; \boldsymbol{\theta})$ parameterized by $\boldsymbol{\theta}$ that takes as input a *latent code* $\mathbf{z}$ and outputs parameters $\mathbf{W}$ that parameterize a *value network* $f(\mathbf{x}; \mathbf{W})$, which then processes the inputs (see Figure 1A). The latent code $\mathbf{z}$ is typically low-dimensional and can be interpreted as a specification of the computation performed by the value network.

Self-attention² maps a sequence of inputs $\mathbf{X} \in \mathbb{R}^{D \times T}$ to a sequence of outputs $\mathbf{Y} \in \mathbb{R}^{D \times T}$. For each attention head $h \in \{1, \dots, H\}$, the inputs are projected into keys $\mathbf{K}_h = \mathbf{W}_h^{\text{key}} \mathbf{X}$ and queries $\mathbf{Q}_h = \mathbf{W}_h^{\text{query}} \mathbf{X}$ using head-specific projection matrices $\mathbf{W}_h^{\text{key}}, \mathbf{W}_h^{\text{query}}, \mathbf{W}_h^{\text{value}} \in \mathbb{R}^{D^{\text{head}} \times D}$ with $D^{\text{head}} = \frac{D}{H}$. The attention matrix can then be obtained as

$$\mathcal{A} = \sigma \left(\begin{bmatrix} \tilde{\mathbf{A}}_1 & \tilde{\mathbf{A}}_2 & \dots & \tilde{\mathbf{A}}_H \end{bmatrix} \right) \text{ with } \tilde{\mathbf{A}}_h = \frac{\mathbf{Q}_h^\top \mathbf{K}_h}{\sqrt{D^{\text{head}}}} \quad (1)$$

where $\mathcal{A} = (a_{h,q,k})_{h,q,k} \in \mathbb{R}^{H \times T \times T}$ stacks the head-specific unnormalized attention matrices $(\tilde{\mathbf{A}}_h)_h$ into a three-dimensional array and applies a normalization operation $\sigma(\cdot)$. For linear attention, $\sigma(\cdot) = \text{Id}(\cdot)$ is simply the identity whereas for softmax attention $\sigma(\cdot) = \text{Softmax}(\cdot)$ applies the softmax operation for each head and query index independently across the key indices.

Considering a single query index $q \in \{1, \dots, T\}$ and $\mathbf{x}_k \in \mathbb{R}^D$ the k -th column of $\mathbf{X}$, multi-head attention (MHA) can be equivalently expressed as a hypernetwork:

$$\text{MHA}_q(\mathbf{X}) := \mathbf{W}^{\text{out}} \bigoplus_{h=1}^H \sum_{k=1}^T a_{h,q,k} \mathbf{W}_h^{\text{value}} \mathbf{x}_k \quad (2)$$

$$= \sum_{h=1}^H \mathbf{W}_h^{\text{out}} \sum_{k=1}^T a_{h,q,k} \mathbf{W}_h^{\text{value}} \mathbf{x}_k \quad (3)$$

$$= \sum_{k=1}^T \left(\sum_{h=1}^H \underbrace{a_{h,q,k}}_{\text{latent code}} \underbrace{\mathbf{W}_h^{\text{out}} \mathbf{W}_h^{\text{value}}}_{\text{hypernetwork}} \right) \mathbf{x}_k \quad (4)$$

$$= \sum_{k=1}^T \underbrace{\mathbf{W}_{q,k}}_{\text{value network}} \mathbf{x}_k \quad (5)$$

where $\oplus$ denotes concatenation and the $\mathbf{W}_h^{\text{out}} \in \mathbb{R}^{D \times D^{\text{head}}}$ are defined as the slices of $\mathbf{W}^{\text{out}} \in \mathbb{R}^{D \times D}$ such that $\mathbf{W}^{\text{out}} = \bigoplus_{h=1}^H \mathbf{W}_h^{\text{out}}$. See Figure 1C for a diagram of the computation.

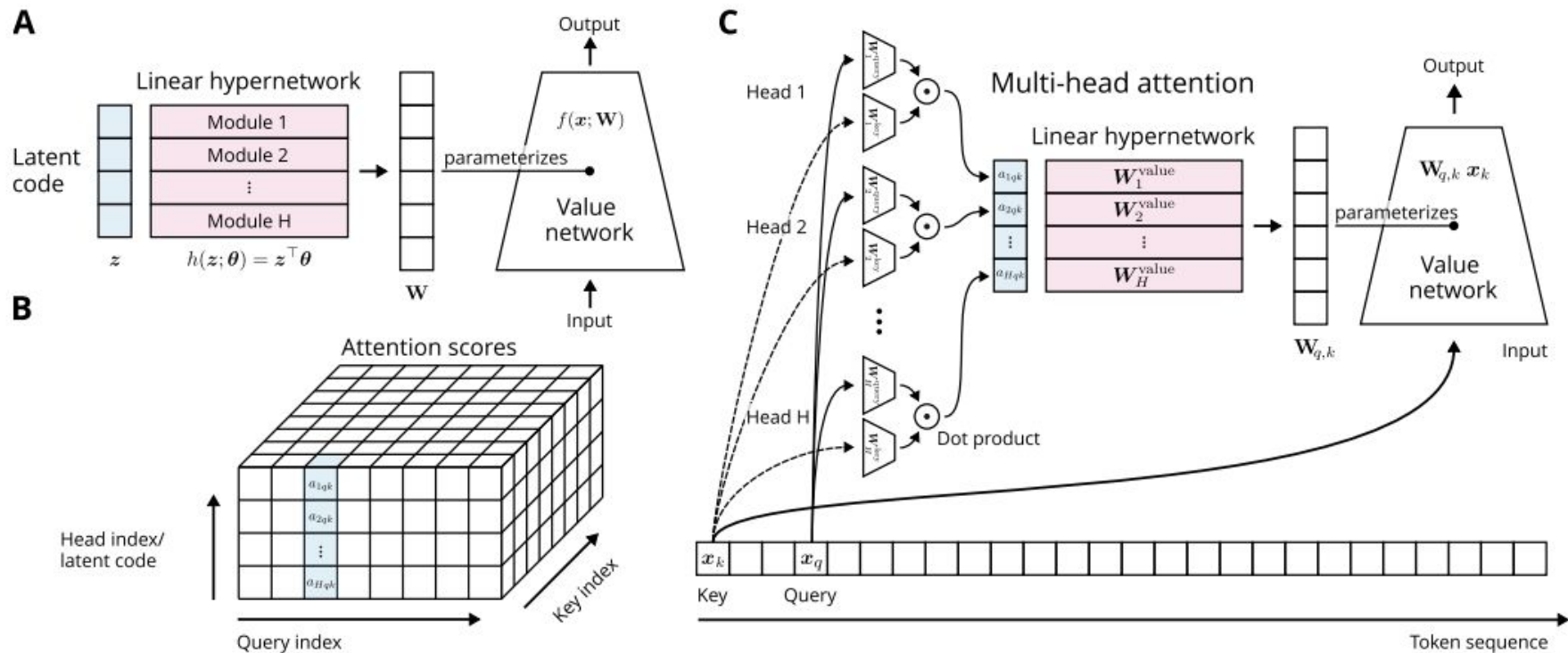


Figure 1: Hypernetwork attention. **A** A linear hypernetwork maps a latent code to a set of parameters that configure a value network to process the input. **B** The attention scores along the head index form the latent code of the hypernetwork. **C** Multi-head attention can be equivalently expressed as a linear hypernetwork that configures key-query specific computations of a linear value network.

$$\text{HYLA}_q(\mathbf{X}) = \sum_{k=1}^T \left(\sum_{h=1}^H a_{h,q,k} \mathbf{W}_h^{\text{out}} \right) \phi \left(\sum_{h=1}^H a_{h,q,k} \mathbf{W}_h^{\text{value}} \mathbf{x}_k \right) \quad (6)$$

$$= \sum_{k=1}^T \mathbf{W}'_{q,k} \phi(\mathbf{W}_{q,k} \mathbf{x}_k), \quad (7)$$

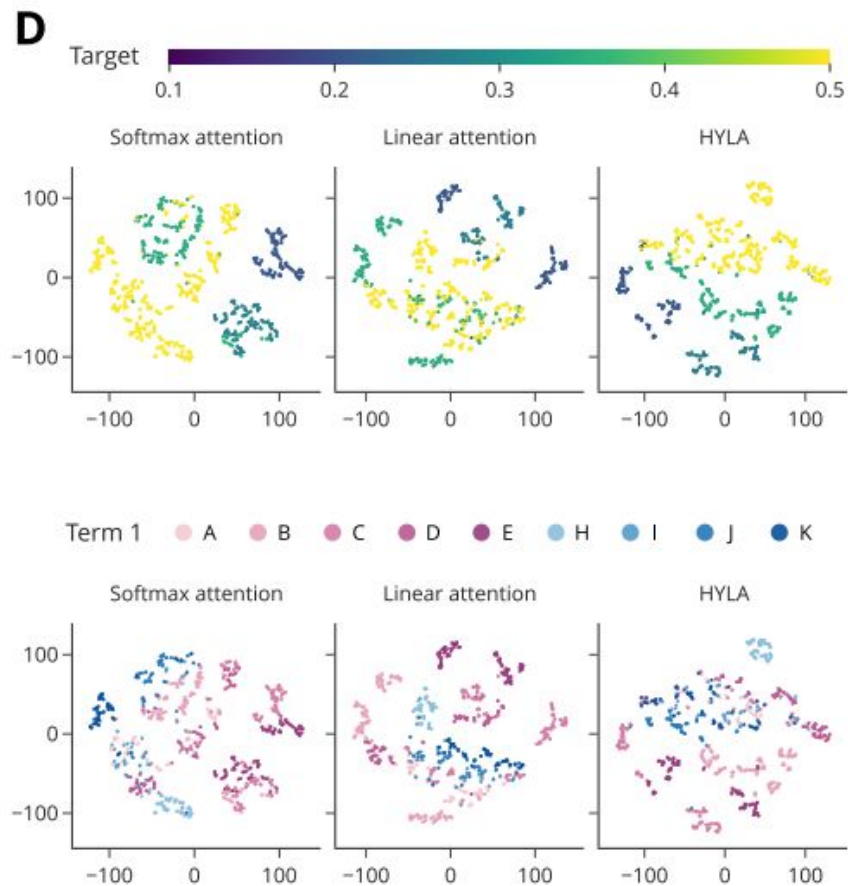
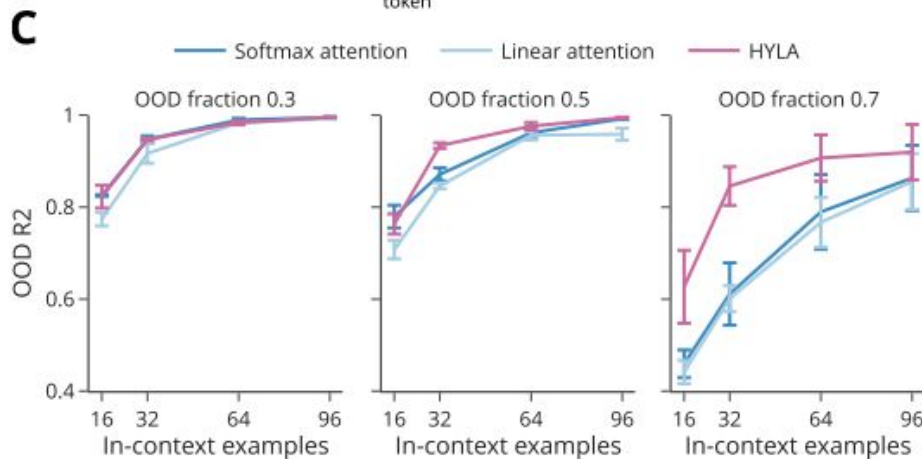
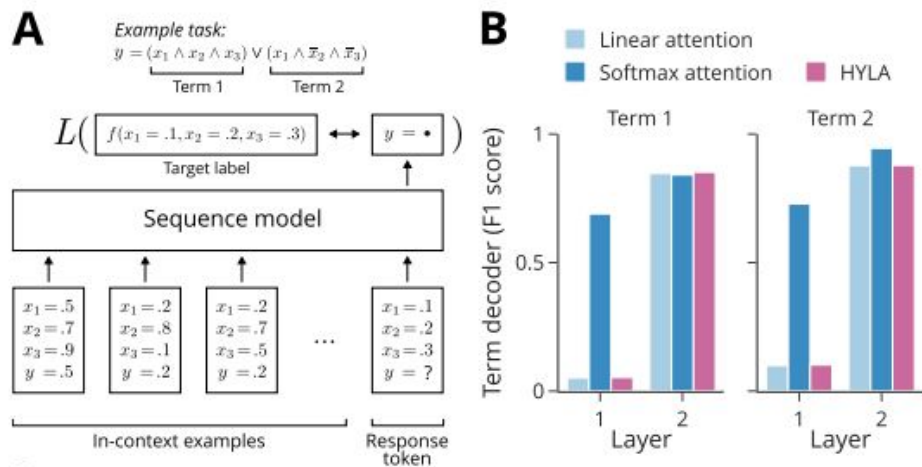
where $\phi(x)$ is an element-wise nonlinearity; here we set it to be a ReLU, $\phi(x) = \max(0, x)$. Additionally, we use $\sigma(\cdot) = \text{RMSHead}(\cdot)$, to normalize the attention scores for each key and query index independently across the head indices using $\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}}$ (without adding

Given a set of L scalars $\{x_1, \dots, x_L\}$ with $x_i \in [0, 1]$ we define fuzzy logic operators on the elements of this set as the Zadeh operators (Zadeh, 1965):

$$x_i \wedge x_j := \min(x_i, x_j), \quad x_i \vee x_j := \max(x_i, x_j), \quad \bar{x}_i := 1 - x_i \quad (8)$$

These operators have the property that for boolean inputs $x_i \in \{0, 1\}$ the outputs coincide with the outputs of the corresponding Boolean operation. Each task is then defined as a function in disjunctive normal form consisting of K terms such as for example for $L = 4$ and $K = 2$

$$f(x_1, x_2, x_3, x_4) = \underbrace{(x_1 \wedge x_2 \wedge x_3 \wedge x_4)}_{\text{Term 1}} \vee \underbrace{(\bar{x}_1 \wedge x_2 \wedge \bar{x}_3 \wedge x_4)}_{\text{Term 2}} \quad (9)$$



A**Split rule combinations****Rules**

- A constant
- B progression (+1)
- C progression (+2)
- D progression (-1)
- E progression (-2)
- F addition
- G difference
- H distribute-three

Splits

Train	AA
	AB
	AD
	⋮
	GH
75%	
OOD	AC
	⋮
	HH
25%	

Sample task rules**Combination of K rules**

- A constant
- B progression (+1)

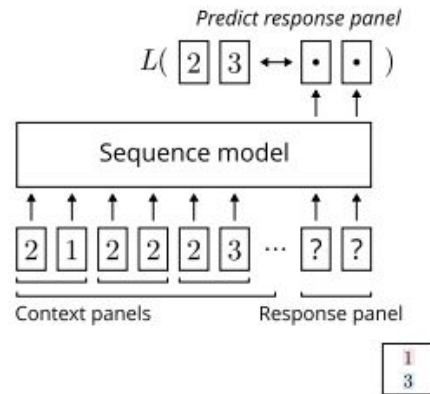
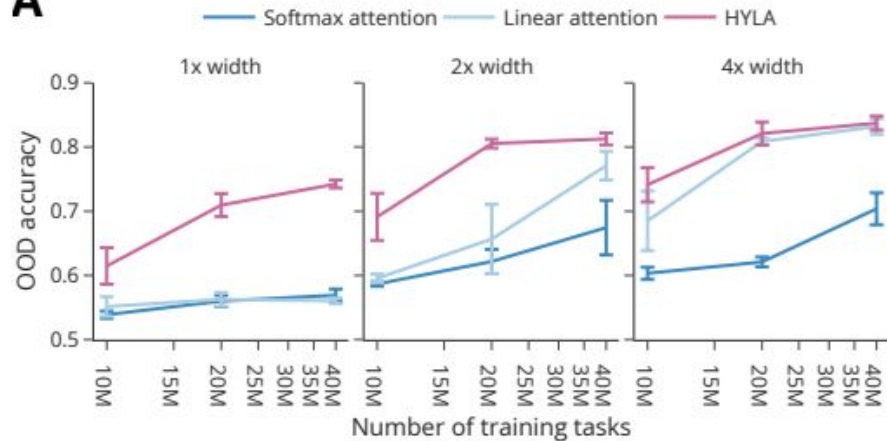
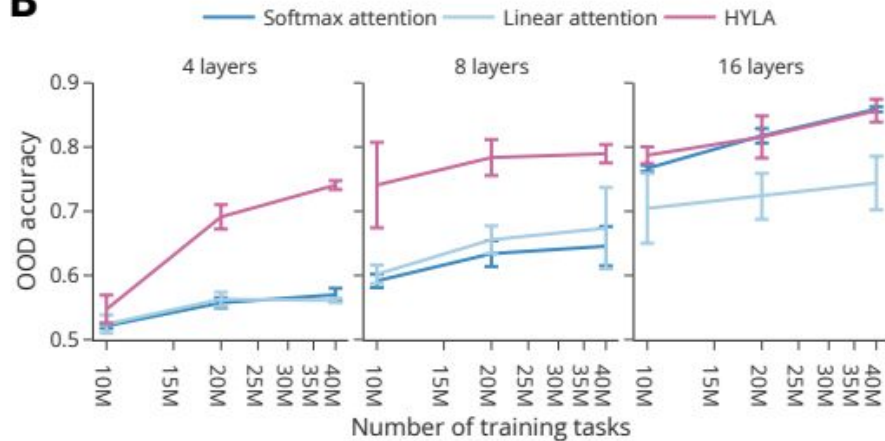
Permutation

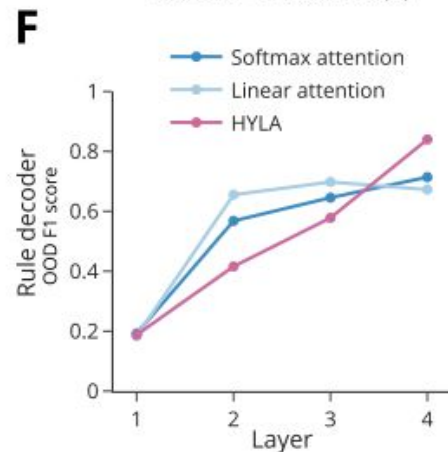
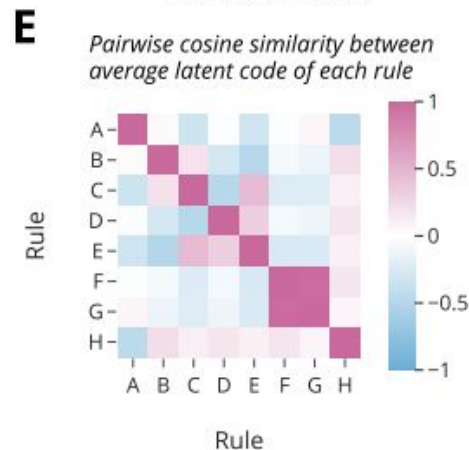
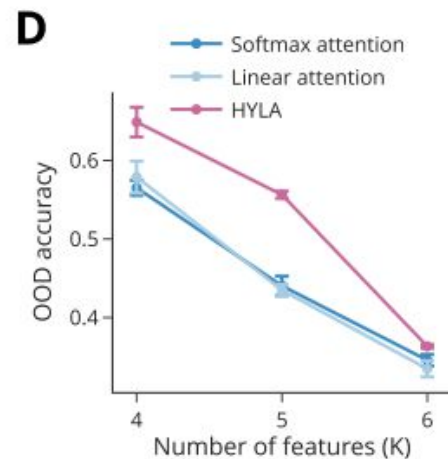
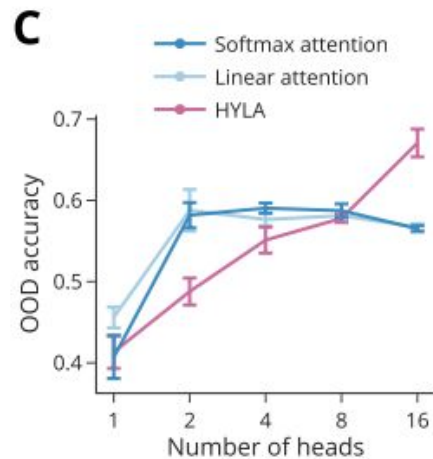
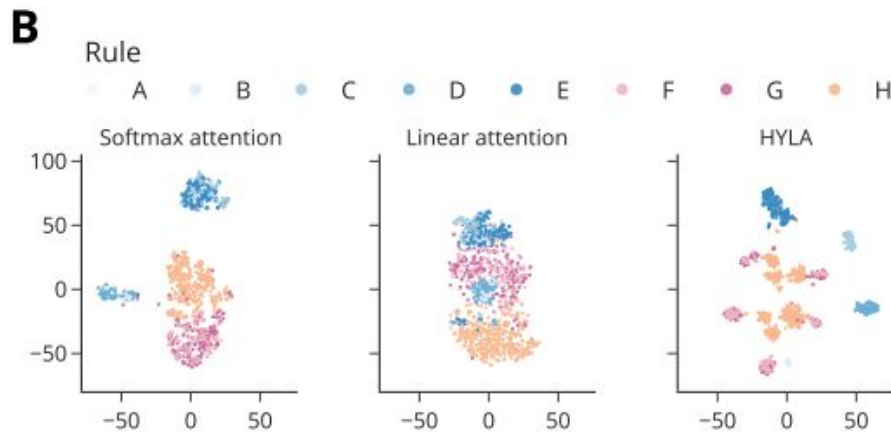
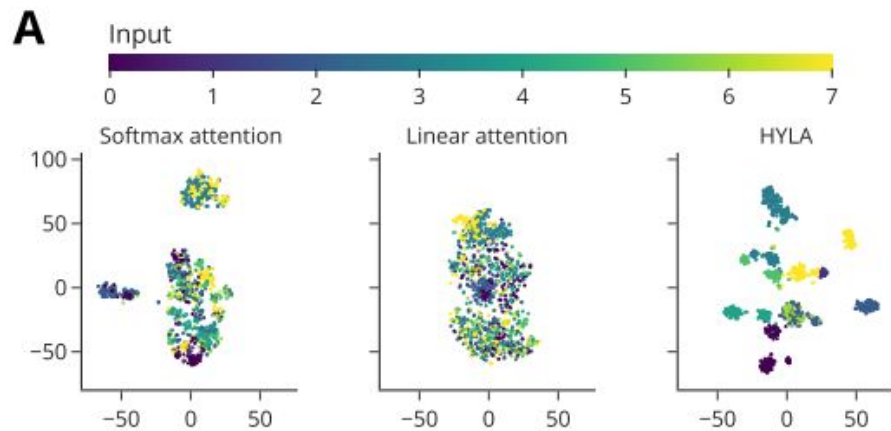
Each column permutes features to model the problem of finding correspondences.

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$
Column 1	Column 2	Column 3

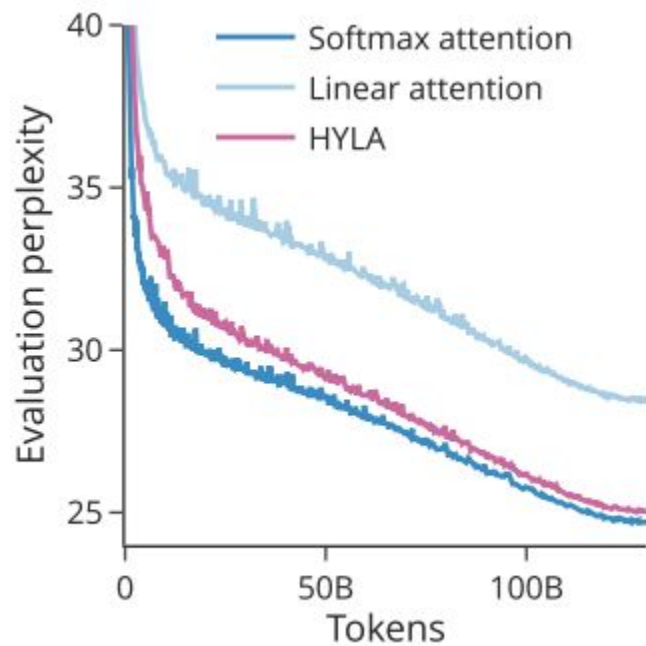
Sample task instance

Context panels	$\begin{bmatrix} 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 3 \end{bmatrix}$
	$\begin{bmatrix} 3 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 4 \end{bmatrix}$
	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} ? \\ ? \end{bmatrix}$
Response panel			

In-context learning**A****B**



LANGUAGE MODELING



Scaling model size and data enables compositional generalization.

Disrupting the hypernetwork mechanism hurts compositional generalization.

The latent code is structured according to the rules of the task.